# Project Executable Files

| | |
|---|---|
| **Date** | 14 July 2026 |
| **Team ID** | |
| **Project Name** | Personalized Networking Assistant |
| **Maximum Marks** | 3 Marks |

### Project Executable Files

This section identifies the executable and supporting files required to run, build, and deploy the **Personalized Networking Assistant**. The repository contains a React and TypeScript frontend, Node.js and Express backend, SQLite persistence, Gemini AI integration, Wikipedia factual verification, environment configuration, build scripts, and academic documentation.

### Step 1: Submission Checklist

| S.No | Item to Submit | Submitted (Yes / No / NA) |
|---|---|---|
| 1 | Complete source code (all files and folders) | Yes |
| 2 | README / Setup Guide with local and production instructions | Yes |
| 3 | package.json and package-lock.json dependency files | Yes |
| 4 | SQLite database configuration and automatic table initialization | Yes |
| 5 | Environment configuration template (.env.example) | Yes |
| 6 | Render deployment configuration / production build support | Yes |
| 7 | APK / desktop executable binary | N/A |
| 8 | Vercel configuration for supported deployment workflow | Yes |
| 9 | PDF validation and documentation generation utilities | Yes |
| 10 | Project documentation and demonstration materials | Yes |

**Step 2: File / Folder Structure**

The repository is organized as a modular full-stack TypeScript application. The structure below highlights the main source, server, API, configuration, documentation, and build files used by the project.

```
Personalized-Networking-Assistant/
|-- 3. Project Design Phase/
|-- 4. Project Planning Phase/
|-- 5. Project Development Phase/
|-- 6. Project Testing/
|-- 7. Project Documentation/
|-- 8. Project Demonstration/
|-- Brainstorming & Ideation/
|-- api/
|-- assets/.aistudio/
|-- server/
|   |-- services/
|-- src/
|   |-- components/
|   |-- types.ts
|-- .env.example
|-- .gitignore
|-- README.md
|-- generate_docs.js
|-- index.html
|-- metadata.json
|-- networking_assistant.json
|-- package.json
|-- package-lock.json
|-- server.ts
|-- tsconfig.json
|-- validate_pdfs.js
|-- vercel.json
`-- vite.config.ts
```

**Step 3: Deployment / Access Details**

The project is configured for production deployment on Render. The production build compiles the frontend into **dist/** and bundles **server.ts** with esbuild into **dist/server.cjs**.

| Field | Details |
|---|---|
| Repository | thishitha06-sketch / Personalized-Networking-Assistant |
| Primary Production Platform | Render |
| Runtime | Node.js |
| Build Command | npm run build |
| Start Command | npm run start |
| Persistent Database | Render persistent disk mounted at /var/data |
| Database Environment Variable | DATABASE_URL=/var/data/networking_assistant.db |

**Production Deployment Configuration**

Connect the repository to Render, select the Node runtime, use **npm run build** as the build command and **npm run start** as the start command. Configure **GEMINI_API_KEY** securely in the environment. For persistent SQLite storage, mount a disk at **/var/data** and set **DATABASE_URL=/var/data/networking_assistant.db**.

**Step 4: Run Instructions**

The following instructions run the full-stack **Personalized Networking Assistant** locally using the Node.js, Express, Vite, and TypeScript development workflow defined in the project repository.

**Pre-requisites**

- Node.js v18 or v20 LTS
- npm
- Internet browser
- Google Gemini API key

**Steps**

1. Clone or download the Personalized Networking Assistant repository.
2. Open the project root folder.
3. Install dependencies: **npm install**
4. Create a **.env** file and configure **GEMINI_API_KEY** and **NODE_ENV=development**.
5. Start the development server: **npm run dev**.
6. Open **localhost:3000** in the browser.
7. For production, run **npm run build** followed by **npm run start**.

**Step 5: Known Issues / Limitations**

| S.No | Known Issue / Limitation | Workaround / Status |
|---|---|---|
| 1 | Gemini generation requires a valid server-side API key and internet connectivity. | Configure GEMINI_API_KEY securely in the environment. The key is never exposed to the client. |
| 2 | Wikipedia factual verification requires network access. | The application performs on-demand server-side lookups when connectivity is available. |
| 3 | SQLite data can reset after a Render redeployment if stored on ephemeral disk. | Attach a persistent disk at /var/data and set DATABASE_URL=/var/data/networking_assistant.db. |